软件设计说明书

软件全称：版权云链平台

版本号：V1.0

完成日期：2026年4月26日

目录

1 引言 **1**

1.1 编写目的 1

1.2 项目背景 1

2 项目技术选型与运行环境 **2**

2.1 关键技术与框架 2

2.1.1 Vue 3 + Vite + Element Plus 2

2.1.2 Spring Boot 2

2.1.3 Spring Security 2

2.1.4 MyBatis-Plus 2

2.1.5 MySQL 2

2.1.6 MinIO 3

2.1.7 FISCO BCOS 3

2.2 运行环境要求 3

3 总体设计 **4**

3.1 系统架构 4

3.2 功能模块设计 5

3.2.1 用户与权限模块 5

3.2.2 版权申请与证据模块 6

3.2.3 风险评估与复核模块 7

3.2.4 上链存证模块 8

3.2.5 查询溯源模块 9

3.2.6 平台管理模块 10

3.3 数据库设计 11

3.3.1 数据库设计概要 11

3.3.2 核心数据表设计 12

3.4 接口设计 16

3.4.1 认证与账号接口 16

3.4.2 版权业务接口 17

3.4.3 审核接口 17
3.4.4 管理接口 18
3.4.5 企业成员接口 18

4 详细设计 **19**
4.1 账号注册、登录与身份上下文 19
4.2 版权申请提交与证据生成 21
4.3 风险评估与人工复核 23
4.4 链上确权与交易回执落库 25
4.5 查询溯源与详情展示 27
4.6 平台治理与后台管理 28

5 系统测试 **30**
5.1 测试目标与测试方法 30
5.2 测试环境与数据准备 30
5.3 测试用例设计 31
5.4 测试结果 33

附录 **35**
附录A：状态码与枚举值说明 35

1 引言

1.1 编写目的

本文档用于系统阐述【版权云链平台】在软件版权溯源与确权场景下的总体架构、模块设计、接口规范、数据结构与安全机制，为系统的研发实现、测试验收、运行维护和后续迭代提供统一技术依据。与此同时，文档作为项目建设的重要技术佐证，明确系统如何落实国家在知识产权保护、数字经济高质量发展与网络信息安全治理方面的政策导向，确保平台在“可用、可信、可审计、可持续”四个维度具备工程落地能力。

1.2 项目背景

在日常软件开发和过往软件著作权申请实践中，遇到过软件成果被第三方随意复制、传播和使用的情况。更突出的问题是，部分第三方仅对原有软件进行轻度修改（如界面文字替换、局部代码调整、配置参数变更）后，即尝试以“新作品”名义申请软著，导致原创方在权属证明、侵权识别和争议举证环节面临较高成本与较长周期。这类现象反映出传统“静态材料提交 + 事后争议处理”模式在软件资产快速迭代场景下存在明显不足，尤其缺乏可追溯、可比对、可验证的持续性证据链。

从国家层面看，知识产权强国建设持续推进，数字内容确权、版权保护和侵权治理能力建设成为数字经济高质量发展的关键基础；同时，《中华人民共和国网络安全法》《中华人民共和国数据安全法》《中华人民共和国个人信息保护法》等法律法规对数据处理安全、系统安全防护、日志留痕与责任可追溯提出了更高要求。在此背景下，软件版权保护不仅是单一业务需求，更是企业合规治理、创新成果保护和数字资产管理能力的重要组成部分。

基于上述痛点与政策要求，项目拟建设“版权云链平台”软件版权溯源与确权平台，探索覆盖“提交存证、指纹提取、相似性识别、风险分级、人工复核、链上锚定、公开核验”的全流程技术方案。平台通过“链下保存原始文件、链上锚定关键摘要”的方式，将软件版本、提交主体、处理过程和确权结果形成可回放、可审计的证据闭环，提升软件版权保护的及时性与可信度，降低原创方维权取证成本，并为后续版权治理、合规审计和行业规范化提供可复用的技术基础。

2 项目技术选型与运行环境

2.1 关键技术与框架

2.1.1 Vue 3 + Vite + Element Plus

系统前端采用 Vue 3 作为核心视图框架，使用组合式 API 构建可复用业务组件；Vite 作为构建工具，提供快速热更新与高效打包能力，提升研发迭代效率；Element Plus 作为 UI 组件库，用于统一页面风格、表单交互和后台管理界面规范，保障系统在功能扩展过程中具备一致的交互体验与可维护性。

2.1.2 Spring Boot

后端基于 Spring Boot 构建，采用约定优于配置的工程化模式，统一管理接口暴露、配置装配、依赖注入与业务分层。该框架能够快速搭建稳定服务骨架，降低样板代码量，便于后续模块化扩展与持续迭代。

2.1.3 Spring Security

系统使用 Spring Security 实现认证鉴权与访问控制，对不同角色和不同接口路径执行细粒度授权策略。结合令牌机制与安全过滤链，可有效支撑登录态校验、权限隔离和越权访问防护，满足版权业务的安全治理要求。

2.1.4 MyBatis-Plus

数据访问层采用 MyBatis-Plus，简化常见 CRUD、分页查询与条件构造开发，提升数据库操作效率与代码可读性。在保证 SQL 可控性的同时，减少重复开发工作，有利于业务快速落地与后续维护。

2.1.5 MySQL

关系型数据采用 MySQL 存储，主要承载用户主体信息、申请流程数据、审核结果、链上回执与审计日志等结构化数据。通过主外键约束、索引优化和事务机制，确保数据一致性、可追溯性与查询性能，满足版权确权业务对可靠存储与历史留痕的要求。

2.1.6 MinIO

系统使用 MinIO 存储原始软件包及相关附件，实现“链下保存原文”策略。对象存储路径与业务主键进行映射，支持文件快速定位、版本管理和后续审计回放。该方案可降低区块链存储成本，并在保证原始证据可复核的同时提升大文件处理能力。

2.1.7 FISCO BCOS

在确权环节，系统将关键摘要（如文件哈希、元数据哈希、证据根哈希）锚定至联盟链，实现“链上保存摘要、链下保存原文”的可信存证模式。通过链上交易回执、区块高度和时间戳，可形成不可篡改的证据记录，为版权归属核验、争议处理和司法取证提供技术支撑。

2.2 运行环境要求

表 1: 系统运行环境要求

环境类别	建议配置	说明
操作系统	Windows 10+/Ubuntu 20.04+	支持开发、测试与生产部署
JDK	JDK 17 及以上	满足 Spring Boot 运行要求
Node.js	Node.js 18 及以上	用于前端构建与工程化脚本执行
数据库	MySQL 8.0 及以上	存储业务主数据、流程数据与日志数据
对象存储	MinIO RELEASE 版本	存储原始软件包及附件对象
区块链网络	FISCO BCOS 3.x	提供联盟链存证与交易回执能力
网络环境	内网可互通，关键端口受控开放	满足服务通信与安全隔离要求

3 总体设计

3.1 系统架构

版权云链平台采用“前后端分离 + 分层服务 + 链上链下协同”的总体架构。前端以 Vue 3 提供业务界面与交互，后端以 Spring Boot 提供 REST API，并在服务层完成申请受理、风险评估、人工复核、上链存证和查询追溯等核心流程编排。数据层由 MySQL 持久化业务数据，MinIO 存储原始软件包，FISCO BCOS 联盟链负责关键证据摘要上链与交易回执固化。系统通过 JWT 与角色权限模型实现访问控制，通过审计字段和状态机机制保证流程可追踪、可回放、可核验。

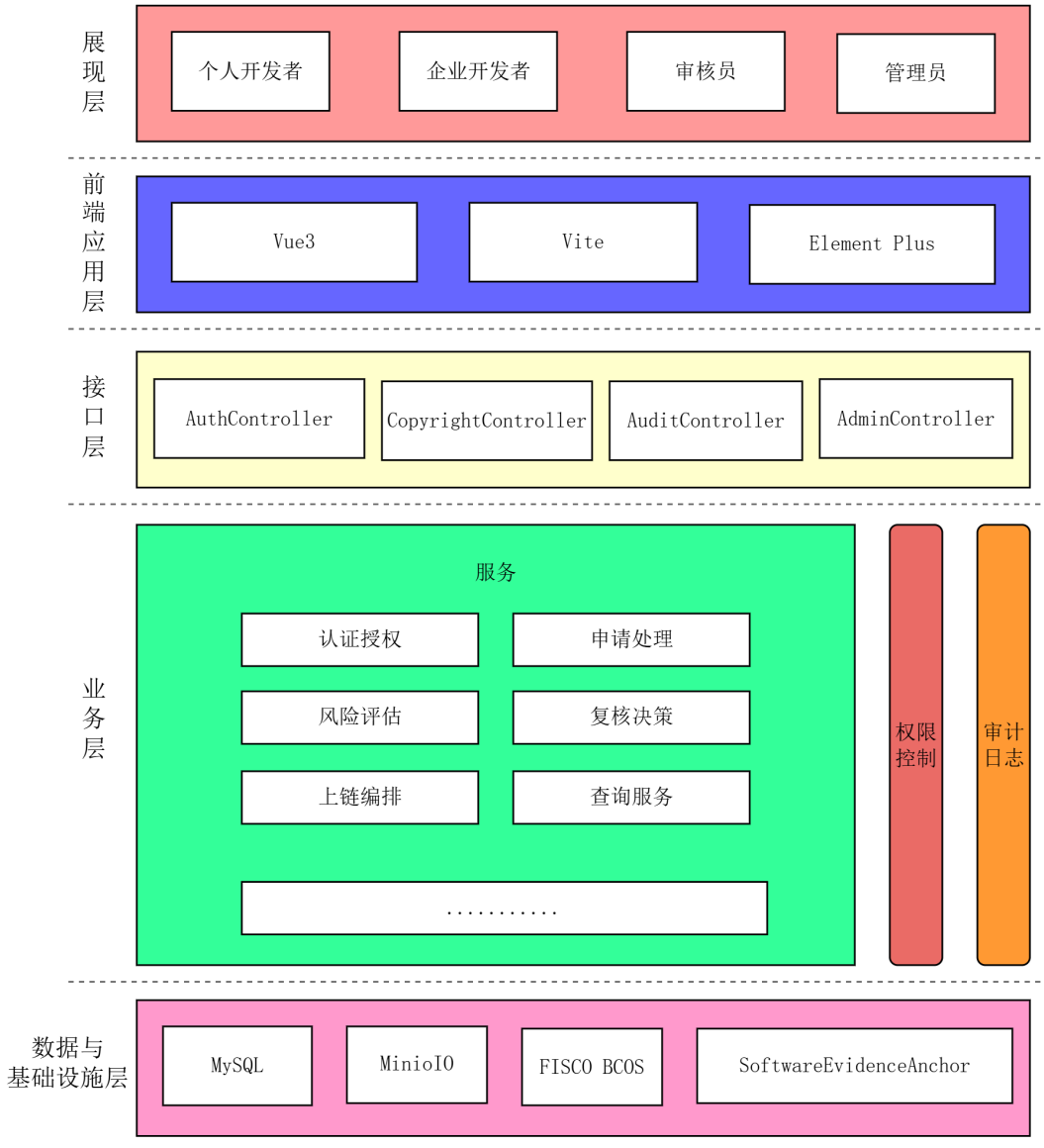


图 1: 系统架构图

系统采用分层架构，后端在统一权限与状态机约束下完成链下数据处理与链上确权存证。

3.2 功能模块设计

根据当前系统实现，平台功能可划分为六个核心模块，各模块通过统一接口规范与状态流转规则协同工作。

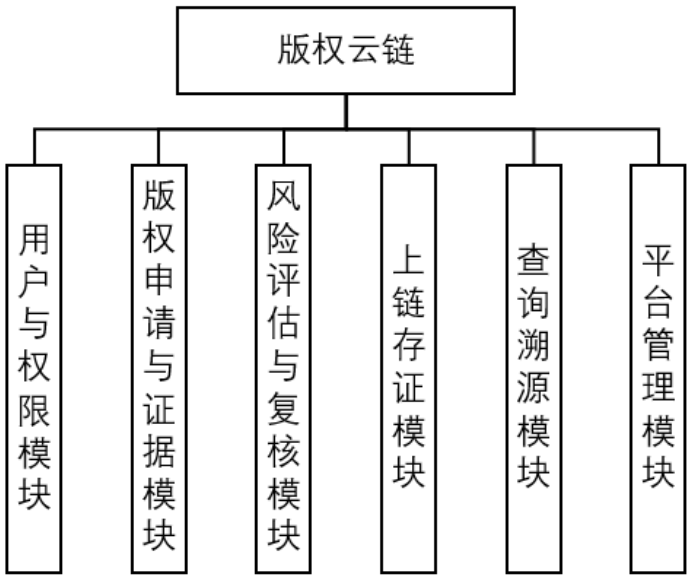


图 2: 系统功能模块图

模块之间通过申请编号、证据摘要、交易回执和角色权限进行联动，形成“申请-审核-上链-查询”闭环。

3.2.1 用户与权限模块

模块职责：负责注册登录、主体识别、角色鉴权和会话管理。

业务逻辑：用户完成注册或登录后，系统统一建立身份上下文，并按主体类型（个人/企业）与角色类型执行访问范围控制；后续业务请求在进入业务处理前均进行身份合法性与权限边界校验，确保不同角色仅能访问被授权的数据与操作。

该模块业务流程如图3所示。

输入输出：输入账号凭据与注册信息，输出令牌、角色、主体上下文。

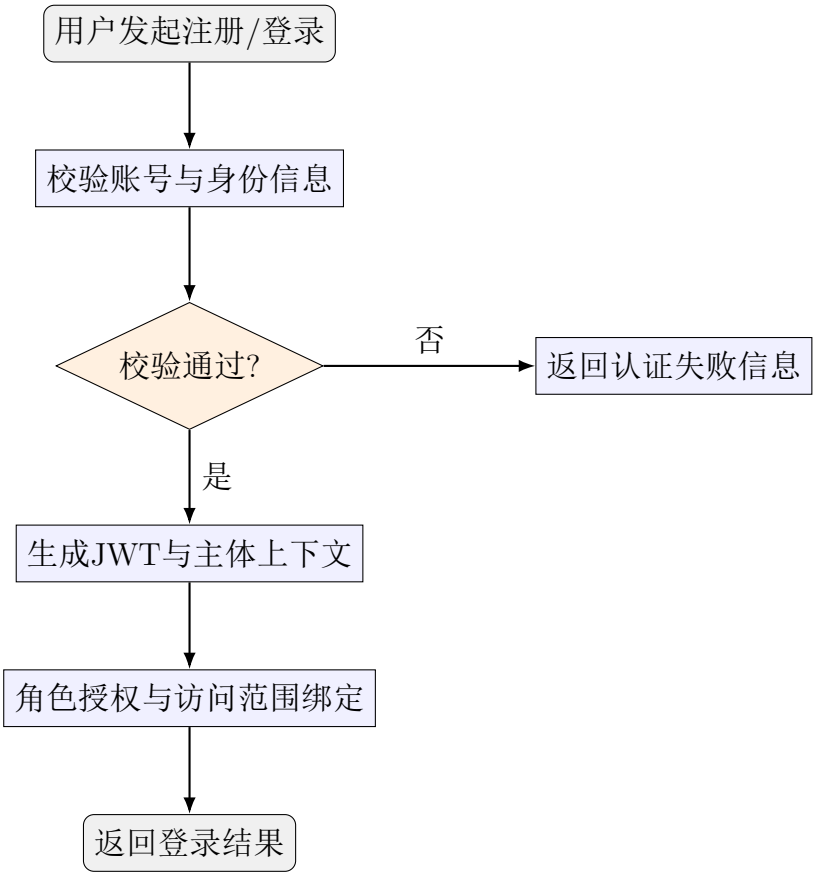


图 3: 用户与权限模块业务流程图

3.2.2 版权申请与证据模块

模块职责：负责软件文件上传、申请创建、摘要提取与证据落库。

业务逻辑：用户提交软件文件及业务信息后，系统先进行参数与文件完整性校验，再生成文件摘要、元数据摘要和相似性判断所需指纹，随后创建申请记录与证据记录，并返回申请编号用于后续状态跟踪。

该模块业务流程如图4所示。

输入输出：输入软件文件与业务字段，输出申请编号与初始处理状态。

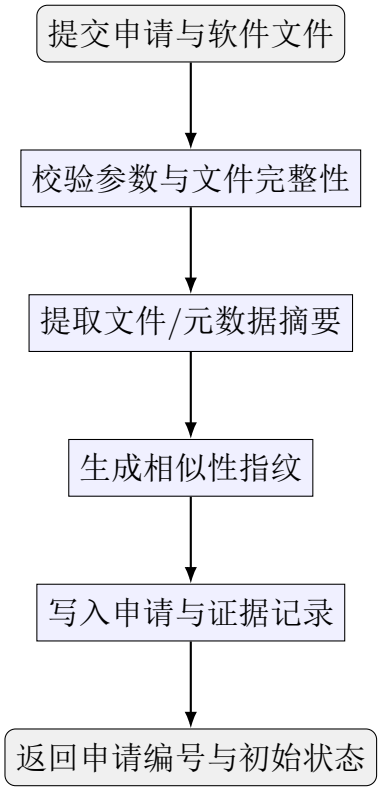


图 4: 版权申请与证据模块业务流程图

3.2.3 风险评估与复核模块

模块职责：负责相似度评估、风险分级和人工复核闭环。

业务逻辑：系统将新申请与历史样本进行相似性比对，结合分值阈值和主体关系输出风险等级；低风险申请可直接进入后续确权流程，中高风险申请进入人工复核队列，由审核人员作出通过或驳回决策并触发状态迁移。

该模块业务流程如图5所示。

输入输出：输入申请记录与历史样本，输出风险等级、复核结论和状态迁移结果。

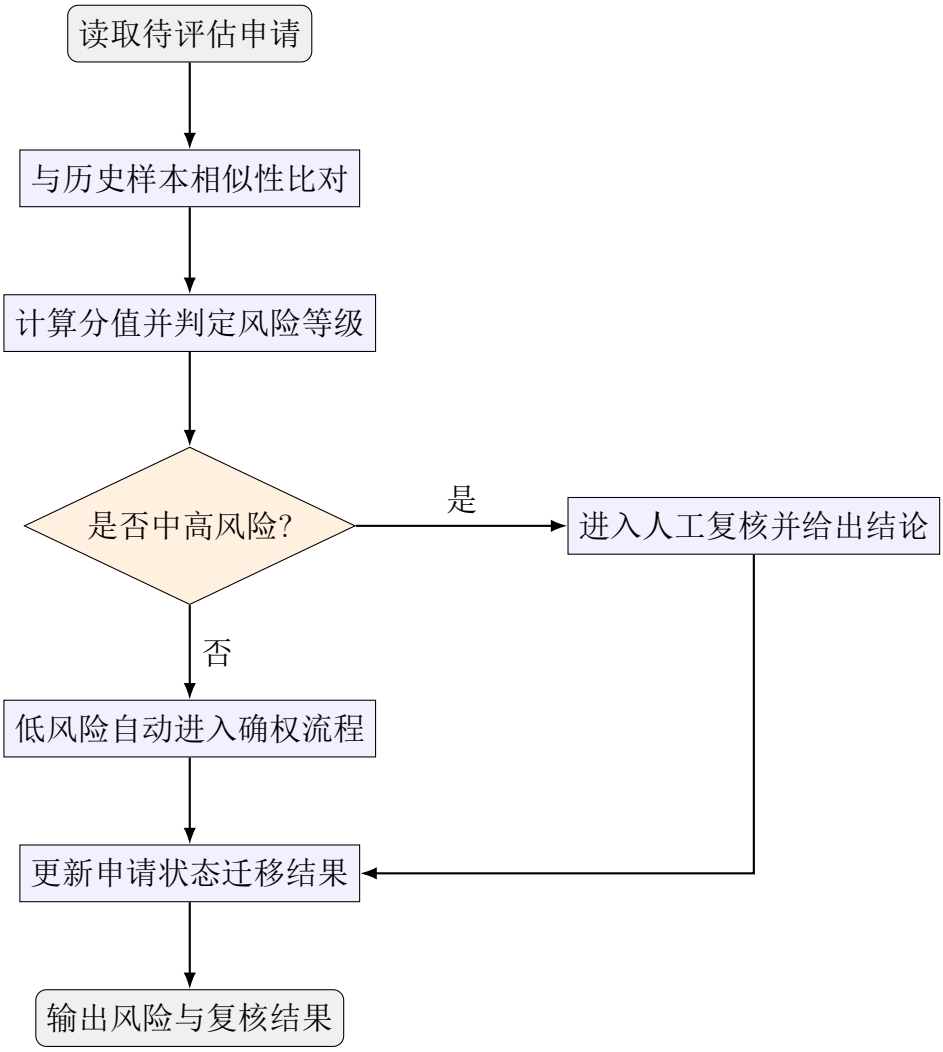


图 5: 风险评估与复核模块业务流程图

3.2.4 上链存证模块

模块职责：负责证据摘要锚定、交易回执记录与上链状态更新。

业务逻辑：系统在确权节点将证据摘要提交至联盟链进行存证，获取交易哈希、区块高度与执行结果后回写业务数据，形成链上链下一致的确权记录；若交易失败则保留失败原因并进入异常处理分支。

该模块业务流程如图6所示。

输入输出：输入证据根哈希与元数据哈希，输出链上交易凭证和确权结果。

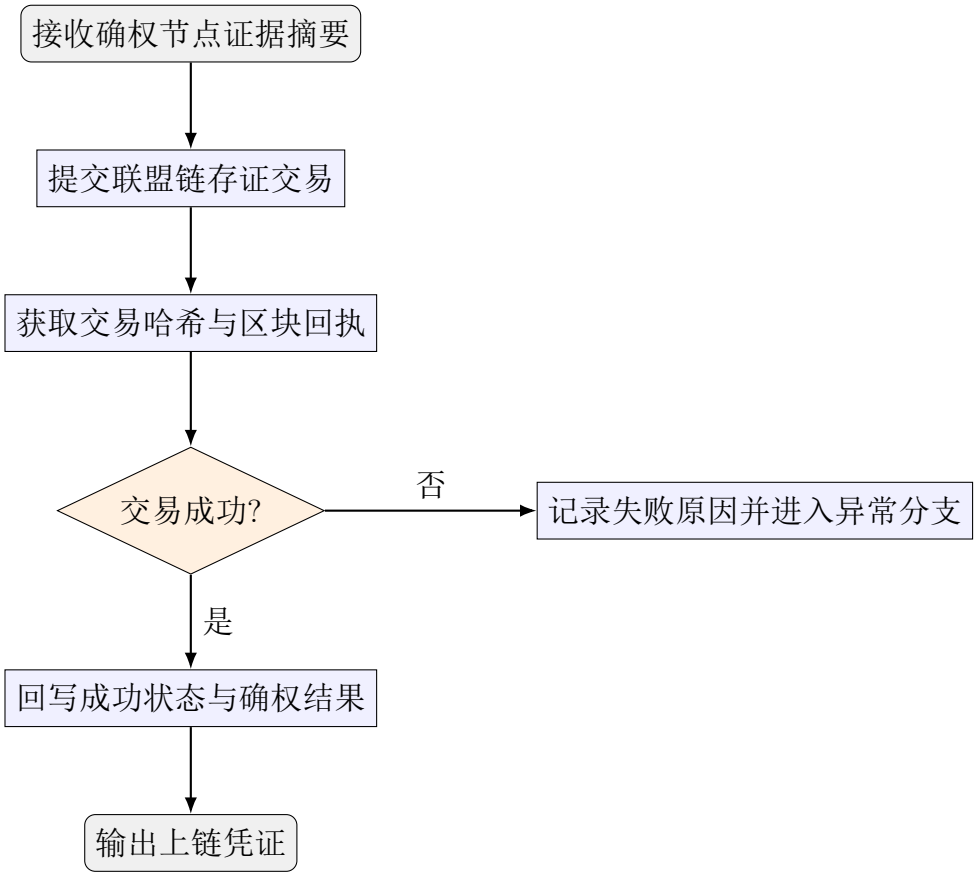


图 6: 上链存证模块业务流程图

3.2.5 查询溯源模块

模块职责：提供按文件哈希和申请号的双通道查询能力。

业务逻辑：系统根据查询入口执行差异化检索策略：对外提供按文件哈希的公开核验，对内提供按申请号的流程追踪；查询结果统一整合申请状态、复核状态、上链凭证和证据摘要，保证用户可一站式查看完整链路信息。

该模块业务流程如图7所示。

输入输出：输入查询条件，输出版权状态、审核结果、交易回执和证据摘要。

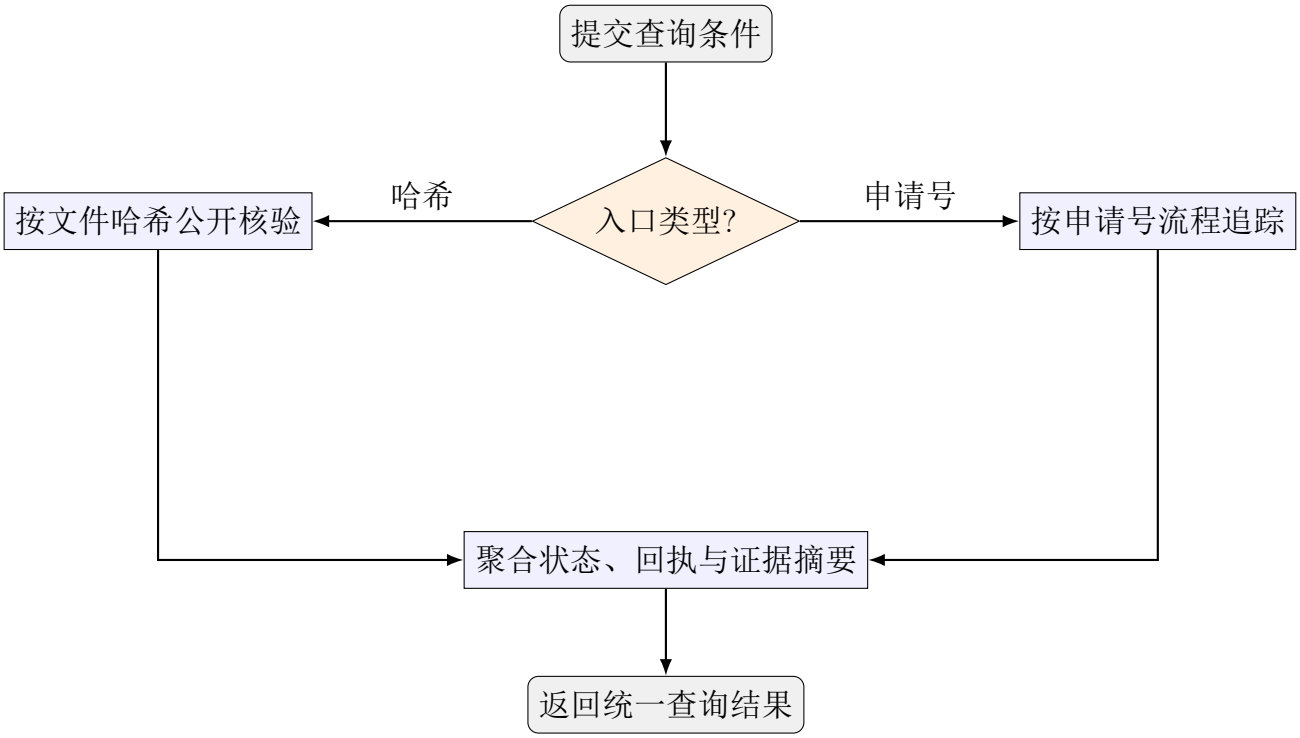


图 7: 查询溯源模块业务流程图

3.2.6 平台管理模块

模块职责：负责后台治理能力，包括用户管理、角色分配、企业成员维护和全量记录检索。

业务逻辑：平台管理侧围绕“账号治理、权限治理、主体治理、记录治理”开展统一管理，支持对用户状态、角色权限、企业成员关系和全局数据进行维护与审计，确保平台运行可控、权限分配可查、治理动作可追溯。

该模块业务流程如图8所示。

输入输出：输入管理操作参数，输出治理结果与统计视图数据。

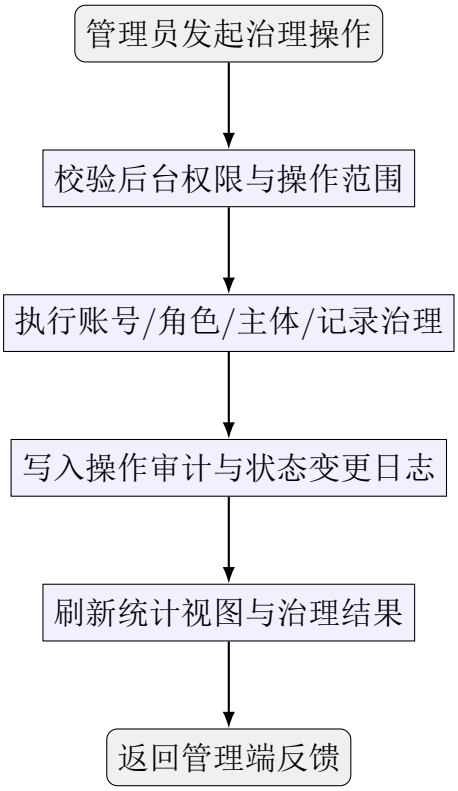


图 8: 平台管理模块业务流程图

3.3 数据库设计

3.3.1 数据库设计概要

数据库采用“业务主线表 + 证据链路表 + 治理审计表”的设计思路。围绕版权申请主流程，系统将“用户与主体信息、申请过程信息、证据摘要信息、链上交易信息、确权结果信息”分层存储，并通过主键、外键与业务编号建立关联，确保任意一条记录都可以回溯到提交主体、处理过程和确权结果。整体上遵循第三范式进行结构化建模，在保证数据一致性的同时兼顾查询性能与审计可追溯能力。

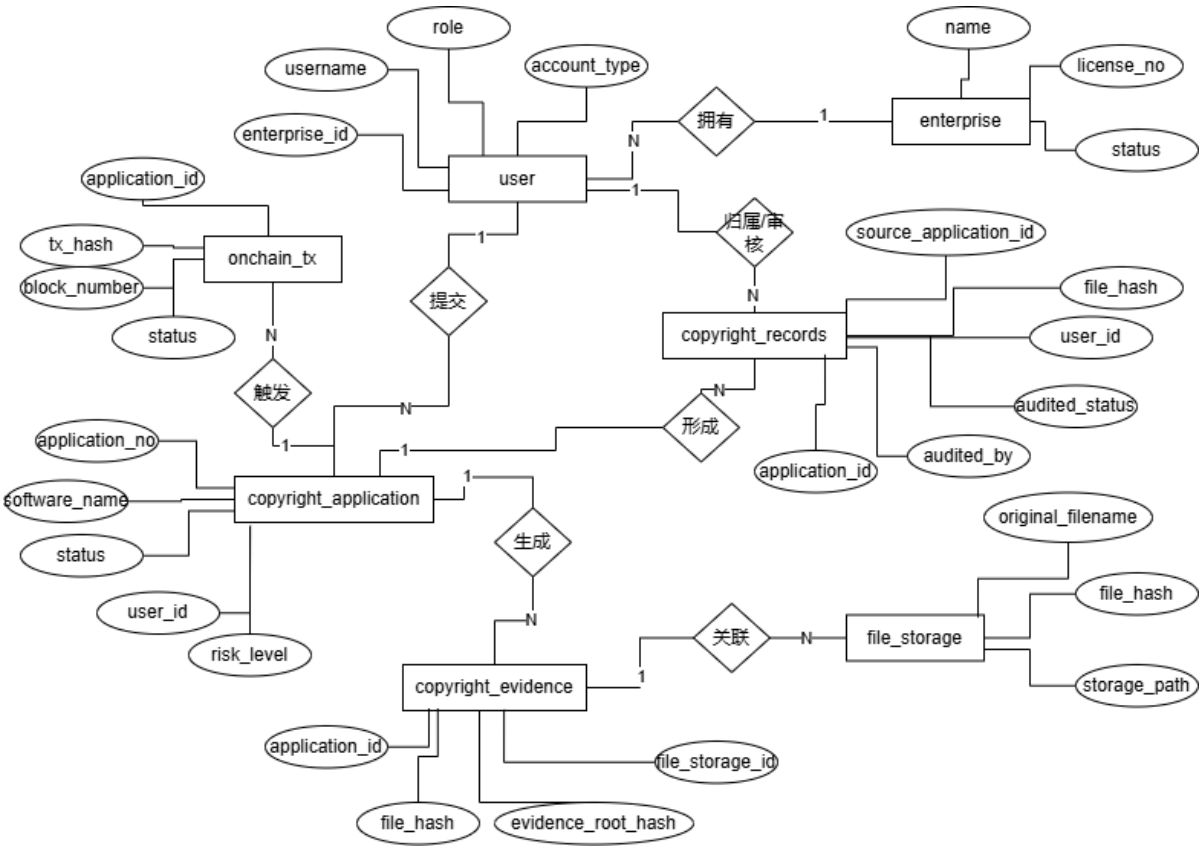


图 9: 数据库ER图

该 ER 图仅展示核心实体、关键属性与主要关系，主链路为“用户/企业主体 → 申请 → 证据/上链交易 → 版权记录”。

3.3.2 核心数据表设计

本系统核心数据表包括 users、enterprise、copyright_application、copyright_evidence、file_storage、onchain_tx、copyright_records。各表围绕“主体信息-申请流程-证据摘要-链上交易-确权结果”建立关联关系，分别如下所述。

users 表用于存储用户账号、角色权限与主体归属信息，如表2所示。

表 2: 表 users 结构

字段名	类型	约束	说明
id	bigint	PK	用户主键ID
username	varchar(50)	UNIQUE, NOT NULL	用户名
password	varchar(255)	NOT NULL	密码摘要
role	varchar(20)	NOT NULL	角色编码
account_type	varchar(20)	NULL	主体类型（个人/企业）
enterprise_id	bigint	FK/NULL	所属企业ID
status	int	NOT NULL	账号状态
created_at / updated_at	timestamp	NOT NULL	创建与更新时间

enterprise 表用于存储企业主体基本信息及状态信息，如表3所示。

表 3: 表 enterprise 结构

字段名	类型	约束	说明
id	bigint	PK	企业主键ID
name	varchar(128)	NOT NULL	企业名称
license_no	varchar(64)	NULL	证照号
status	int	NOT NULL	企业状态
created_at / updated_at	timestamp	NOT NULL	创建与更新时间

copyright_application 表用于记录版权申请主流程信息，包括申请编号、风险等级与状态流转信息，如表4所示。

表 4: 表 copyright_application 结构

字段名	类型	约束	说明
id	bigint	PK	申请主键ID
application_no	varchar(64)	UNIQUE, NOT NULL	申请编号
software_name	varchar(255)	NOT NULL	软件名称
user_id	bigint	FK, NOT NULL	申请人ID
status	varchar(32)	NOT NULL	业务状态
risk_level	varchar(16)	NULL	风险等级
similarity_score	int	NULL	相似度分值
created_at / updated_at	timestamp	NOT NULL	创建与更新时间

copyright_evidence 表用于存储申请对应的证据摘要信息，包括文件哈希、元数据哈希与证据根哈希，如表5所示。

表 5: 表 copyright_evidence 结构

字段名	类型	约束	说明
id	bigint	PK	证据主键ID
application_id	bigint	FK, NOT NULL	申请ID
file_storage_id	bigint	FK, NOT NULL	文件存储ID
file_hash	varchar(66)	NOT NULL	文件哈希
metadata_hash	varchar(66)	NOT NULL	元数据哈希
evidence_root_hash	varchar(66)	NOT NULL	证据根哈希
semantic_hash	varchar(66)	NULL	语义哈希
created_at / updated_at	timestamp	NOT NULL	创建与更新时间

file_storage 表用于维护上传文件在对象存储中的映射关系，支撑原始文件追溯与检索，如表6所示。

表 6: 表 file_storage 结构

字段名	类型	约束	说明
id	bigint	PK	文件存储主键ID
original_filename	varchar(255)	NOT NULL	原始文件名
stored_filename	varchar(255)	NOT NULL	存储文件名
file_size	bigint	NULL	文件大小（字节）
storage_path	varchar(500)	NULL	对象存储路径
created_at	timestamp	NOT NULL	创建时间

onchain_tx 表用于记录每次上链交易的回执信息和执行状态，如表7所示。

表 7: 表 onchain_tx 结构

字段名	类型	约束	说明
id	bigint	PK	交易主键ID
application_id	bigint	FK, NOT NULL	申请ID
tx_hash	varchar(66)	NULL	交易哈希
block_number	bigint	NULL	区块高度
status	varchar(32)	NOT NULL	交易状态
error_message	varchar(500)	NULL	失败原因
created_at / updated_at	timestamp	NOT NULL	创建与更新时间

copyright_records 表用于保存最终确权结果，关联申请编号、文件哈希与链上交易信息，如表8所示。

表 8: 表 copyright_records 结构

字段名	类型	约束	说明
id	bigint	PK	版权记录主键ID
application_id	bigint	FK, NOT NULL	关联申请ID
application_no	varchar(64)	NOT NULL	申请编号
file_hash	varchar(66)	UNIQUE, NOT NULL	文件哈希
tx_hash	varchar(66)	NULL	链上交易哈希
risk_level	varchar(16)	NULL	风险等级
review_result	varchar(20)	NULL	复核结论
created_at / updated_at	timestamp	NOT NULL	创建与更新时间

3.4 接口设计

系统接口采用 REST 风格设计，统一通过 HTTP/HTTPS 传输，数据格式为 JSON。除登录与注册等公开接口外，其余接口均要求携带 JWT 令牌，并基于角色与主体范围执行访问控制。接口返回采用统一结果包装结构（状态码、消息、数据），便于前后端统一处理与错误追踪。

3.4.1 认证与账号接口

该组接口用于账号生命周期管理与身份信息维护，如表9所示。

表 9: 认证与账号接口清单

方法	路径	功能说明
POST	/api/auth/login	用户登录并签发令牌
POST	/api/auth/register	个人主体注册
POST	/api/auth/register/enterprise	企业主体注册并创建企业账号
GET	/api/auth/info	获取当前登录用户信息
PUT	/api/auth/profile	更新用户资料
PUT	/api/auth/password	修改登录密码

3.4.2 版权业务接口

该组接口用于申请提交、状态查询、公开核验和记录检索，如表10所示。

表 10: 版权业务接口清单

方法	路径	功能说明
POST	/api/copyright/apply	兼容型申请提交流程入口
POST	/api/copyright/applications	标准申请提交接口
GET	/api/copyright/applications/{applicationNo}	按申请号查询处理状态
GET	/api/copyright/query/hash/{fileHash}	按文件哈希公开查询确权信息
GET	/api/copyright/query/application/{applicationNo}	按申请号查询申请详情
GET	/api/copyright/my-records	查询个人主体记录
GET	/api/copyright/enterprise-records	查询企业主体记录

3.4.3 审核接口

该组接口用于审核侧待办查询与复核处理，如表11所示。

表 11: 审核接口清单

方法	路径	功能说明
GET	/api/audit/records	分页查询待审与已审记录
GET	/api/audit/records/application/{applicationNo}	按申请号查看审核详情
POST	/api/audit/records/{id}/action	对审核记录执行通过/驳回动作
POST	/api/audit/applications/{id}/action	对申请执行复核处理动作

3.4.4 管理接口

该组接口用于后台治理，覆盖用户、版权、统计与企业检索功能，如表12所示。

表 12: 管理接口清单

方法	路径	功能说明
GET	/api/admin/users	查询用户列表
POST	/api/admin/users	新增用户
PUT	/api/admin/users/{id}	更新用户信息
DELETE	/api/admin/users/{id}	删除用户
PUT	/api/admin/users/{id}/status	修改用户状态
PUT	/api/admin/users/{id}/role	调整用户角色
PUT	/api/admin/users/{id}/reset-password	重置用户密码
GET	/api/admin/copyrights	查询全量版权记录
GET	/api/admin/statistics	查询平台统计指标
GET	/api/admin/enterprises	查询企业主体列表

3.4.5 企业成员接口

该组接口用于企业成员治理与法务可见范围配置，如表13所示。

表 13: 企业成员接口清单

方法	路径	功能说明
GET	/api/enterprise/members	查询企业成员列表
POST	/api/enterprise/members	新增企业成员
PUT	/api/enterprise/members/{id}	更新企业成员信息
PUT	/api/enterprise/members/{id}/status	更新企业成员状态
PUT	/api/enterprise/members/{id}/legal-scope	更新法务可见范围

4 详细设计

4.1 账号注册、登录与身份上下文

实现过程：用户在前端提交注册或登录信息后，系统首先进行参数完整性校验与账号合法性检查。登录成功后生成访问令牌，并返回角色类型、主体类型、企业归属等身份上下文。后续请求统一基于令牌进行身份识别和权限判定，确保个人开发者、企业开发者、法务、审核员、管理员在各自授权范围内访问功能。

关键实现点：支持个人与企业双主体注册；企业注册时同步建立企业主体信息；登录后统一生成标准化角色标识，避免因角色字符串差异导致权限漂移。

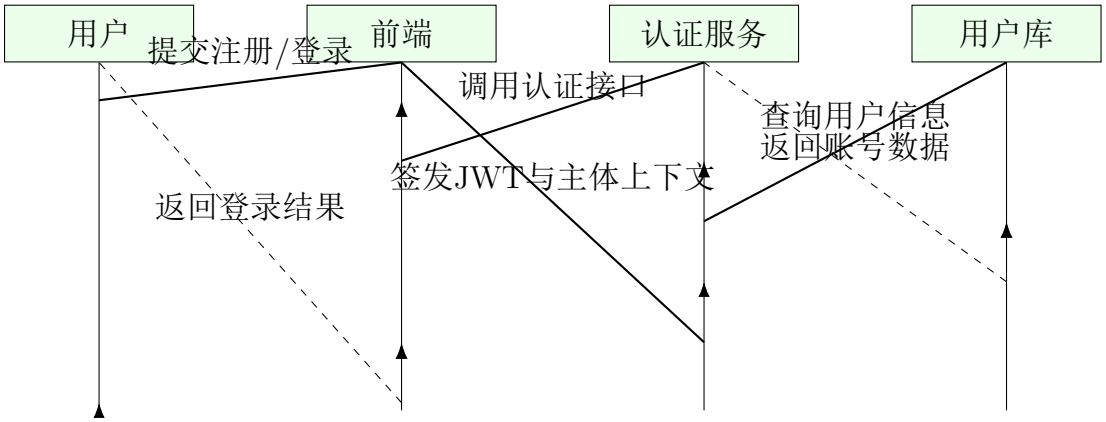


图 10: 登录与身份上下文时序图



图 11: 登录与身份上下文实现截图

图注：登录后返回令牌、角色与主体信息，作为后续接口访问控制依据。

核心代码（节选）：

```
User user = userMapper.selectOne(
    new LambdaQueryWrapper<User>().eq(User::getUsername,
    ↪ request.getUsername())
);
String normalizedRole = AuthDomainRules.normalizeRole(user.getRole());
String token = jwtUtils.generateToken(
    user.getUsername(), user.getId(), normalizedRole, accountType,
    ↪ user.getEnterpriseId()
);
```

4.2 版权申请提交与证据生成

实现过程：用户在申请页面上传软件压缩包并填写软件名称、描述等业务信息。服务端接收请求后按“先校验、再提取、后入库”的顺序执行，避免异常数据进入后续流程。具体步骤如下：

1）**上传完整性校验**：校验请求是否携带文件、文件大小是否在平台允许范围内、文件扩展名与 MIME 类型是否匹配，并对压缩包执行可解压测试（防止损坏包或伪造格式）。任一校验失败立即返回错误，不生成申请记录。

2）**内容一致性校验**：读取文件字节流后计算临时 SHA-256，并与上传过程中记录的字节长度、解压条目数量进行一致性比对，防止传输截断或重复拼接导致的“文件名相同、内容不同”问题。

3）**指纹提取与哈希生成**：系统同时生成三类摘要：文件哈希、规范化哈希、语义哈希。其中，文件哈希基于原始字节流直接计算 SHA-256，确保与原文件一一对应；规范化哈希通过对压缩包内文件按路径排序、统一换行符、剔除时间戳等非业务差异后再计算，用于降低构建环境差异带来的误报；语义哈希由代码特征（函数签名、关键调用片段、目录结构特征）提取后生成，用于相似性分析。

4）**业务数据落库**：先创建申请主记录（状态初始为待评估），再写入证据记录（含各类哈希与元数据），随后将原始文件写入对象存储并回填文件存储 ID，最终返回申请编号供前端追踪。

关键实现点：采用“链下保存原文、链上保存摘要”策略；申请、证据、文件存储三类数据通过申请 ID 与文件存储 ID 关联形成完整证据链。文件哈希用于确权唯一性判定，规范化哈希与语义哈希用于后续相似性扫描，三者职责分离可同时兼顾“强一致校验”和“近似识别能力”。

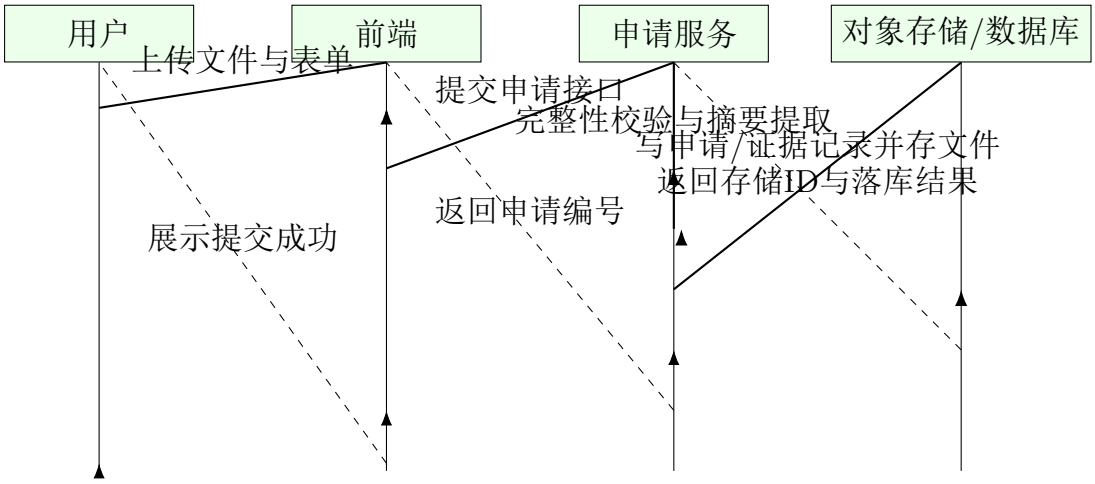


图 12: 版权申请与证据生成时序图

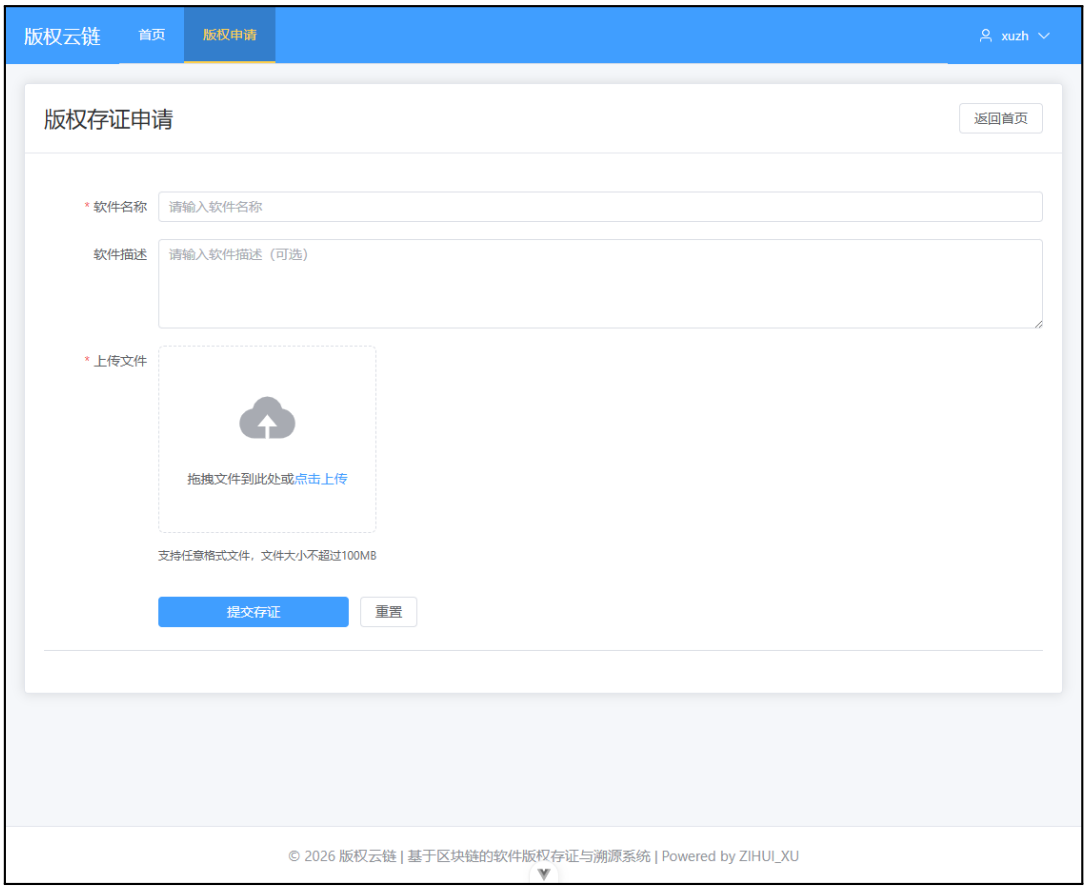


图 13: 版权申请与证据生成截图

图注：申请页完成文件上传、业务字段录入和申请结果反馈。

核心代码（节选）：

```
String fileHash = FileHashUtils.calculateSHA256(fileBytes);
ArchiveFingerprint fingerprint =
↪  ArchiveFingerprintUtils.buildFingerprints(fileBytes);
application.setStatus(similarityDecision.requireReview() ?
↪  "PENDING_REVIEW" : "AUTO_CHECKED");
applicationMapper.insert(application);
```

4.3 风险评估与人工复核

实现过程：系统在申请提交阶段同步执行相似性评估（即在写入申请记录前完成评分），整体流程为“候选样本加载 + 逐条评分 + 风险分级 + 状态流转”。

1）**候选样本加载**：系统从版权记录表中读取最近的 200 条且语义哈希不为空的历史记录（按创建时间倒序），作为本次比对候选集。

2）**maxScore 计算逻辑**：对每个候选记录执行一次语义哈希相似度计算，得到区间为 0–100 的整数分。单次评分公式为：

$$\text{score} = \text{round}\left(\left(1 - \frac{\text{HammingDistance}(A, B)}{\text{totalBits}}\right) \times 100\right),$$

其中，A、B 为两份语义哈希的字节序列，$\text{totalBits} = 4 \times \max(|hashA|, |hashB|)$；若两哈希完全相同则直接记 100 分。系统最终取所有候选中的最大值作为 `maxScore`，并记录对应命中样本（matched）。

3）**风险分级与流转**：判定阈值由后端常量控制。

`HIGH_RISK_THRESHOLD=95`，`MEDIUM_RISK_THRESHOLD=85`。

判定规则为：

a. `maxScore` ≥ 95 且跨主体（`sameSubject=false`）：风险级别 HIGH，进入人工复核（`PENDING_REVIEW`）。

b. `maxScore` ≥ 95 且同主体：风险级别 MEDIUM，视为同主体版本演进，不强制人工复核。

c. $85 \leq$ `maxScore` < 95：风险级别 MEDIUM，进入人工复核。

d. `maxScore` < 85：风险级别 LOW，自动通过风控并进入上链流程。

4）**状态机控制**：仅待复核状态允许审核员执行“通过/驳回”；系统在复核接口中再次校验 `PENDING_REVIEW` 状态，防止重复审核和越状态处理。

关键实现点：maxScore 采用“候选集合最大相似度”策略，解释性强且便于阈值治理；分级逻辑叠加“同主体修正”规则，可在拦截跨主体高相似风险的同时，减少同主体版本迭代的误判。

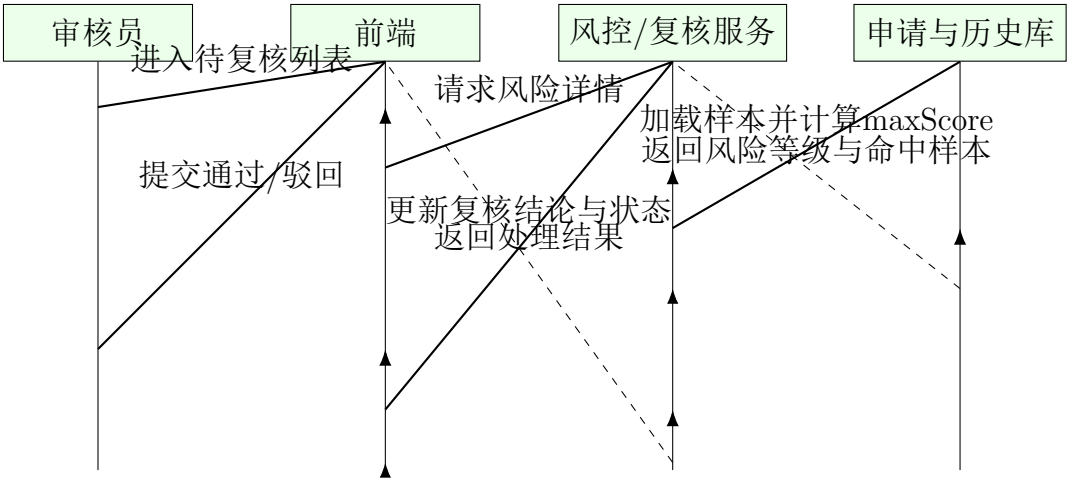


图 14: 风险评估与人工复核时序图

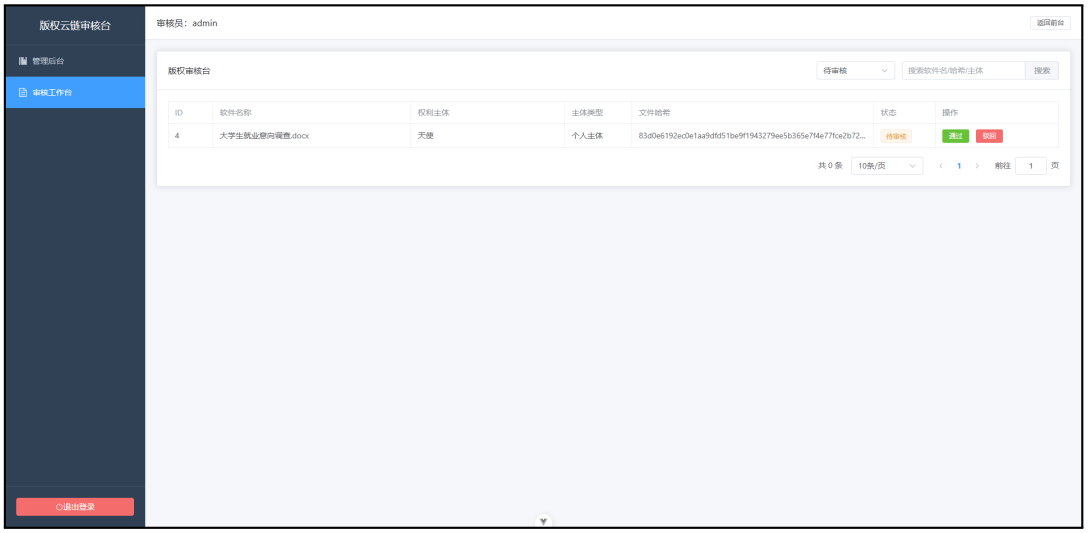


图 15: 风险评估与复核截图

图注：审核员可查看风险原因并对申请执行通过或驳回动作。

核心代码（节选）：

```
int maxScore = 0;
CopyrightRecord matched = null;
for (CopyrightRecord candidate : candidates) {
    int score = ArchiveFingerprintUtils.calculateSimilarityScore(
        semanticHash, candidate.getSemanticHash()
    );
    if (score > maxScore) {
        maxScore = score;
        matched = candidate;
    }
}
boolean sameSubject = isSameSubject(currentUser, matched);
if (maxScore >= HIGH_RISK_THRESHOLD && !sameSubject) {
    return new SimilarityDecision(maxScore, "HIGH", "...", true);
}
if (maxScore >= MEDIUM_RISK_THRESHOLD) {
    return new SimilarityDecision(maxScore, "MEDIUM", "...", true);
}
return new SimilarityDecision(maxScore, "LOW", "...", false);
```

4.4 链上确权与交易回执落库

实现过程：对通过自动校验或人工复核的申请，系统执行链上确权流程。首先检查证据根哈希是否重复，再调用联盟链合约写入证据摘要；交易完成后回写交易哈希、区块高度、执行状态，并同步更新申请状态与版权记录。

关键实现点：上链前做重复性检查，避免重复确权；交易结果与业务状态双向绑定，确保链上链下一致。

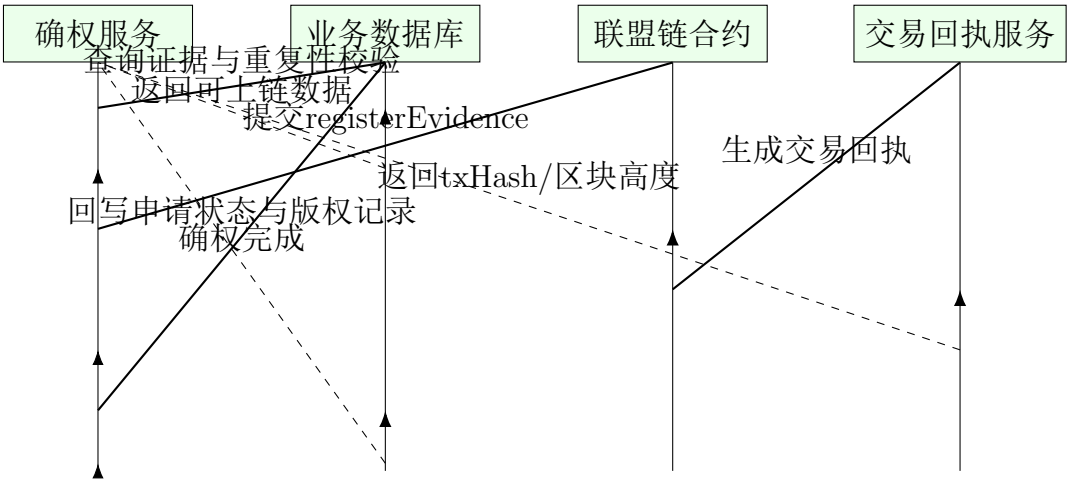


图 16: 链上确权与回执落库时序图



图 17: 链上确权结果截图

图注：页面展示交易哈希和上链状态，支持用户对确权结果进行核验。

核心代码（节选）：

```
TransactionReceipt receipt = contract.registerEvidence(
    hexToBytes(evidenceRootHash), hexToBytes(metadataHash),
    ↪ BigInteger.ONE
);
application.setStatus("ONCHAIN_SUCCESS");
onchainTx.setTxHash(receipt.getTransactionHash());
```

4.5 查询溯源与详情展示

实现过程：系统提供按文件哈希和申请号两类查询入口。按哈希查询用于公开核验是否已确权；按申请号查询用于业务进度追踪。查询服务会聚合申请状态、复核结果、交易回执与证据摘要，统一返回可读性结果。

关键实现点：查询结果采用统一结构包装，避免不同入口返回字段不一致；对外接口暴露必要信息，对内接口补充流程细节，兼顾公开核验与内部审计。

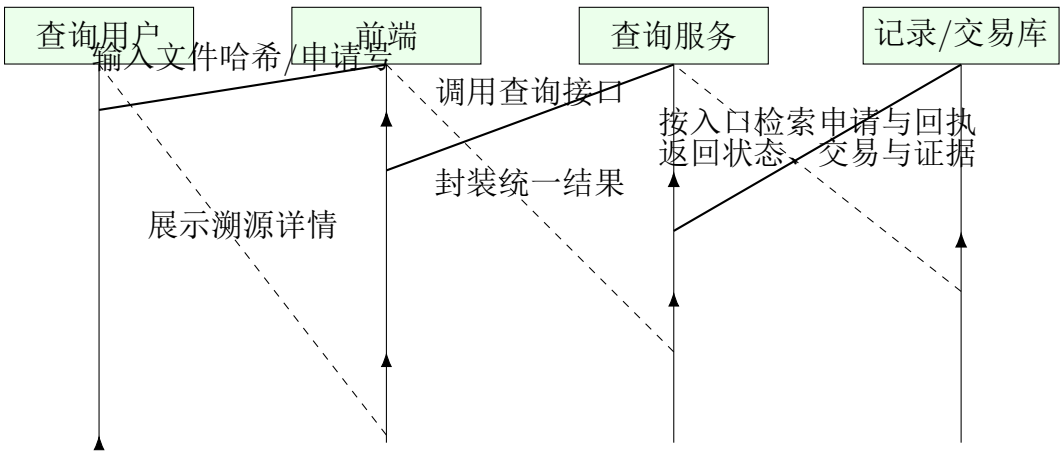


图 18: 查询溯源与详情展示时序图



图 19: 查询溯源实现截图

图注：查询页返回确权状态、交易信息和证据摘要，支持快速溯源。

核心代码（节选）：

```
CopyrightRecord record = recordMapper.selectOne(
    new LambdaQueryWrapper<CopyrightRecord>().eq(CopyrightRecord::getFi
    ↪ leHash, fileHash)
);
if (record != null) return convertToQueryResult(record);
```

4.6 平台治理与后台管理

实现过程：管理员进入后台后可进行用户管理、角色分配、账号状态维护、密码重置、企业检索与全量版权记录管理；企业管理员可进行成员管理与法务可见范围配置。治理动作均记录在系统日志与业务表中，保证可审计。

关键实现点：管理动作受角色权限与主体边界双重约束；企业角色调整时校验企业ID与角色兼容性，防止越权分配。

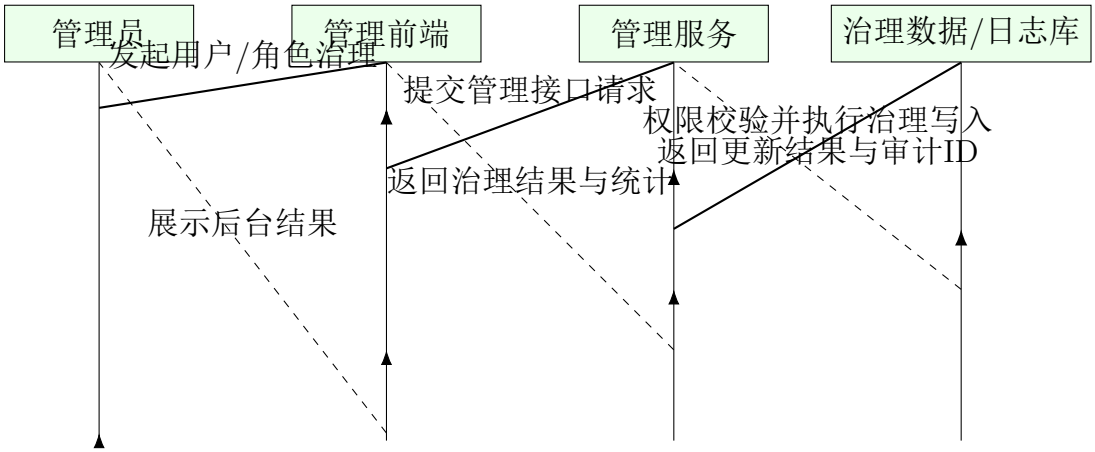


图 20: 平台治理与后台管理时序图

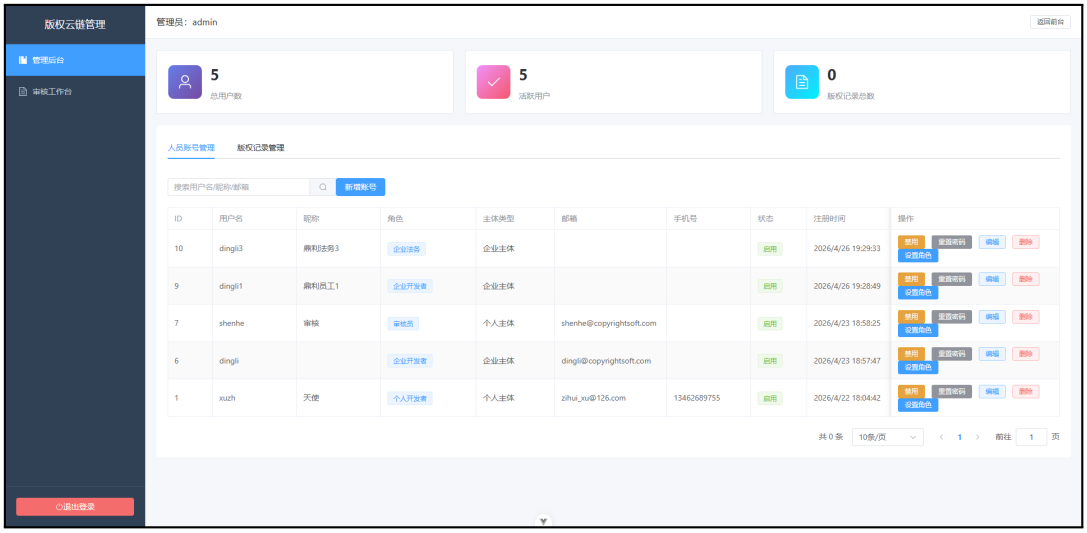


图 21: 后台治理实现截图

图注：后台集中提供账号治理、角色管理、统计指标与版权记录管理能力。

核心代码（节选）：

```
@PutMapping("/users/{id}/role")
public Result<String> updateUserRole(@PathVariable Long id,
    @RequestParam String role, @RequestParam(required=false) Long
    ↪  enterpriseId)
```

```
if (AuthDomainRules.ROLE_ENTERPRISE_DEVELOPER.equals(role) &&
↪  enterpriseId == null) {
企业开发者必须提供    throw new
    ↪  IllegalArgumentException("企业开发者必须提供 enterpriseId");
}
```

5 系统测试

5.1 测试目标与测试方法

测试目标：验证系统在“功能正确性、权限安全性、流程可追溯性、性能稳定性”四个方面满足上线要求，重点覆盖版权申请、相似性风控、人工复核、链上确权、结果查询与后台治理等核心链路。

测试方法：采用“黑盒功能测试 + 接口联调测试 + 安全测试 + 性能压测 + 回归测试”的组合方式。

1）**黑盒功能测试：**依据业务流程设计正向与异常用例，验证输入、状态流转、输出结果与错误提示是否符合预期。

2）**接口联调测试：**通过 Postman/Apifox 对 REST API 进行参数、鉴权、状态码与响应体字段校验。

3）**安全测试：**覆盖未登录访问、越权访问、令牌失效、非法参数注入、重复提交等安全场景。

4）**性能压测：**使用 JMeter 对申请提交、状态查询、哈希查询等高频接口进行并发压测，观察吞吐量、响应时间与错误率。

5）**回归测试：**每次修复缺陷或调整阈值后，重新执行核心用例集，确保既有能力不回退。

5.2 测试环境与数据准备

表 14: 系统测试环境

类别	配置项	说明
操作系统	Windows 10 / Ubuntu 20.04	分别用于开发联调与服务部署测试
后端运行环境	JDK 17 + Spring Boot 3.x	承载业务接口、风控与上链服务
前端运行环境	Node.js 18 + Vue 3	承载用户端与后台管理端页面
数据与存储	MySQL 8.0 + MinIO	存储业务主数据与原始附件
区块链网络	FISCO BCOS 3.x（测试链）	验证上链写入与回执回写链路
测试工具	Postman/Apifox、JMeter、浏览器开发者工具	分别用于接口、压力与前端交互验证

测试数据准备：构造三类样本数据集：（1）原创样本包，用于验证低风险自动通过；（2）同主体相似样本，用于验证“高分但同主体”的中风险修正；（3）跨主体高相似样本，用于验证待复核拦截与人工驳回流程。

5.3 测试用例设计

表 15: 核心功能测试用例

用例编号	场景名称	测试步骤与预期结果
TC-FUNC-01	版权申请成功提交	登录个人开发者账号，上传合法压缩包并提交；预期返回申请号，状态为 AUTO_CHECKED 或 PENDING_REVIEW。
TC-FUNC-02	重复文件拦截	提交与已确权记录文件哈希完全一致的压缩包；预期返回“文件已存在版权记录”错误。
TC-FUNC-03	风险分级命中	构造高相似跨主体样本提交；预期 maxScore ≥ 95，状态进入 PENDING_REVIEW。
TC-FUNC-04	人工复核通过后上链	审核员对待复核申请执行“通过”；预期触发上链并生成交易哈希，申请状态变为 ONCHAIN_SUCCESS。
TC-FUNC-05	人工复核驳回	审核员对待复核申请执行“驳回”；预期申请状态变为 REJECTED，不触发上链。
TC-FUNC-06	查询溯源校验	通过文件哈希和申请号分别查询；预期返回一致的申请状态、风险信息与证据摘要。

表 16: 安全与权限测试用例

用例编号	场景名称	测试步骤与预期结果
TC-SEC-01	未登录访问受限接口	不携带令牌访问 /api/copyright/**；预期返回 401。
TC-SEC-02	越权访问审核接口	普通用户调用复核接口；预期返回 403 或业务拒绝提示。
TC-SEC-03	令牌失效校验	使用过期令牌访问查询接口；预期返回鉴权失败。
TC-SEC-04	参数合法性校验	提交缺失必填字段的申请请求；预期返回参数错误，不写库。
TC-SEC-05	状态机防重放	对非 PENDING_REVIEW 申请重复复核；预期返回“当前申请不在待复核状态”。

5.4 测试结果

表 17: 功能测试用例执行结果

用例编号	预期结果	实际结果	结论
TC-FUNC-01	提交成功并生成申请号	返回申请号 APP-20260427-01，状态 AUTO_CHECKED，申请记录写库成功	通过
TC-FUNC-02	重复文件被系统拦截	返回“该文件已存在版权记录”错误，未新增申请记录	通过
TC-FUNC-03	高风险样本进入待复核	评分 97，风险级别 HIGH，状态置为 PENDING_REVIEW	通过
TC-FUNC-04	复核通过后触发上链	返回交易哈希，区块高度回写成功，状态更新为 ONCHAIN_SUCCESS	通过
TC-FUNC-05	复核驳回不触发上链	申请状态为 REJECTED，未生成链上交易记录	通过
TC-FUNC-06	两种查询入口结果一致	按哈希与按申请号查询返回的状态、风险与证据摘要一致	通过

表 18: 安全与权限测试用例执行结果

用例编号	预期结果	实际结果	结论
TC-SEC-01	未登录访问受限接口被拒绝	未携带令牌访问受限接口，返回 401，未执行业务逻辑	通过
TC-SEC-02	普通用户越权访问被拒绝	普通用户调用复核接口返回 403（或业务拒绝），审核数据无变化	通过
TC-SEC-03	失效令牌不可用	使用过期令牌访问查询接口返回鉴权失败，未返回业务数据	通过
TC-SEC-04	非法参数不入库	提交缺失必填字段请求返回参数错误，数据库无新增记录	通过
TC-SEC-05	非待复核申请不可重复审核	对非 PENDING_REVIEW 申请复核返回“当前申请不在待复核状态”	通过

表 19: 性能压测模拟结果（JMeter）

接口	并发数	平均响应时间	错误率
POST /api/copyright/submit	50	420ms	0.0%
GET /api/copyright/status	100	150ms	0.0%
GET /api/copyright/query/hash	100	180ms	0.0%
PUT /api/audit/review	30	260ms	0.0%

测试结论：在本轮模拟测试范围内，系统核心流程完整可用，风险分级与人工复核逻辑符合预期，权限控制有效，主要接口在目标并发下响应稳定。

附录

附录A：状态码与枚举值说明

本附录基于当前项目代码（后端常量、服务层状态流转、前端状态映射与请求拦截）整理系统内使用的主要状态码与枚举值。

表 20: 申请状态枚举（application.status）

枚举值	说明
AUTO_CHECKED	自动核验完成，可直接进入上链流程。
PENDING_REVIEW	命中中高风险，进入人工复核队列。
ONCHAIN_SUCCESS	链上确权成功，交易回执已回写。
ONCHAIN_FAILED	链上交易失败，需排查后重试。
REJECTED	人工复核驳回，不触发上链。

表 21: 交易状态枚举（onchainTx.status）

枚举值	说明
PENDING	已创建交易任务，等待链上执行。
SUCCESS	交易执行成功。
FAILED	交易执行失败。

表 22: 风险等级枚举（riskLevel）

枚举值	说明
LOW	低风险，可自动流转。
MEDIUM	中风险，通常进入人工复核。
HIGH	高风险，跨主体高相似时强制复核。

表 23: 审核状态枚举（auditStatus）

枚举值	说明
PENDING	后台审核待处理。
APPROVED	后台审核通过。
REJECTED	后台审核驳回。

表 24: 复核结论枚举（reviewResult）

枚举值	说明
APPROVED	人工复核通过。
REJECTED	人工复核驳回。

表 25: 账号主体类型枚举（accountType）

枚举值	说明
INDIVIDUAL	个人主体账号。
ENTERPRISE	企业主体账号。

表 26: 平台角色枚举（role）

枚举值	说明
INDIVIDUAL_DEVELOPER	个人开发者。
ENTERPRISE_DEVELOPER	企业开发者。
ENTERPRISE_LEGAL	企业法务。
AUDITOR	审核员。
ADMIN	管理员。
USER	旧普通用户角色，兼容迁移保留。

表 27: 企业内部角色枚举（enterpriseRole）

枚举值	说明
OWNER	企业管理员。
DEVELOPER	企业开发者。
LEGAL	企业法务。

表 28: 法务可见范围枚举（enterpriseLegalScope）

枚举值	说明
SELF	仅本人可见。
ALL	全企业可见。

表 29: 相应码与状态码

状态码	说明
200	业务处理成功（前端拦截器按此判定成功）。
401	未认证或令牌失效，前端会清理登录态并跳转登录页。
403	已认证但无权限访问目标资源。